

SYSTEM ARCHITECTURE DOCUMENT V2.0

DIY Robot Challenge 2026 — Full Architecture

Complete technical architecture for an autonomous differential-drive outdoor obstacle-course robot. Jetson Orin Nano, Hesai QT64, ACEINNA MTLT335 IMU, Trimble BX992 RTK, and ROS2 Humble.

COMPETITION

October 1–2, 2026

VERSION

2.0 — May 2026

DRIVE TYPE

Differential (2×
Kraken)

ROS2 Humble

FAST-LIO2

Trimble BX992 RTK

Nav2

Hesai QT64

RealSense D435i

ACEINNA MTLT335

Raspberry Pi 4B

Table of Contents

1 Overview

1.1 Purpose

1.2 Scope

1.3 Key Stakeholders

1.4 Definitions & Assumptions

2 Architecture Overview

2.1 Architecture Style

2.2 Key Design Principles

2.3 Constraints & Tradeoffs

3 System Context Diagram

4 Component Architecture

4.1 Hardware Components

4.2 Software Component Diagram

4.3 Component Descriptions

5 Sequence Diagrams

5.1 Normal Navigation Flow

5.2 Obstacle Zone Entry

5.3 E-Stop Activation

6 Data Architecture

6.1 Data Flow Diagram

6.2 Storage & Configuration

6.3 ROS2 Topic Map

7 Deployment Architecture

8 Non-Functional Requirements

9 Risks & Mitigations

10 Development Timeline

1.1 Purpose

This System Architecture Document (SAD) describes the complete hardware and software architecture for an autonomous differential-drive robot competing in the **Caterpillar Culture Club DIY Robot Challenge 2026**. It serves as the single source of truth for all design decisions, integration interfaces, and implementation strategy.

1.2 Scope

IN SCOPE	OUT OF SCOPE
Full autonomous navigation stack	Mechanical chassis design (pre-built)
Sensor integration & calibration	Battery management system internals
Obstacle detection & behavior switching	Manual/RC driving mode
Pre-competition mapping pipeline	Speed course strategy (separate doc)
Safety/E-Stop architecture	Post-competition data analysis tools

1.3 Key Stakeholders

ROLE	RESPONSIBILITY
Software Lead	ROS2 stack development, Nav2 configuration, FAST-LIO2 integration
Hardware Lead	Sensor mounting, CAN wiring, E-Stop integration, power distribution
Firmware Lead	Raspberry Pi CAN bridge, motor control, E-Stop handler
Systems Lead	Calibration, testing, competition day operations

1.4 Definitions & Assumptions

Definitions

- **FAST-LIO2** — Fast LiDAR-Inertial Odometry using iterated EKF
- **EKF** — Extended Kalman Filter (state estimation algorithm)
- **RTK** — Real-Time Kinematic GPS correction
- **Nav2** — ROS2 Navigation2 stack
- **MPPI** — Model Predictive Path Integral controller
- **PCL** — Point Cloud Library
- **SM** — State Machine (Mission State Machine = Mission SM)
- **Diff-drive** — Two-motor differential drive (skid-steer)
- **DDS** — Data Distribution Service (ROS2 middleware layer)
- **URDF** — Unified Robot Description Format (defines TF tree)
- **CAN** — Controller Area Network (automotive bus protocol)

Assumptions

- Competition is outdoors — RTK has open sky for majority of course
- Trimble BX992 is dual-antenna → provides GPS heading
- ACEINNA MTLT335 internal Kalman outputs corrected orientation at 100Hz
- Robot weight is within 25lbs including all added sensors
- Track surface: grass, gravel, pavement mix
- Obstacles are placed ≥ 1 day before competition (Rule 3.2)
- Raspberry Pi 4B can achieve CAN communication via MCP2515 SPI hat

1.5 Competition Constraints

PARAMETER	VALUE	NOTES
Max dimensions	16" W × 24" L × 16" H	Hard limit — never exceed during run
Max weight	25 lbs	Including all electronics & battery
Battery	12V / 15Ah	~180Wh available. Monitor consumption.
Obstacle course	2 consecutive laps	Timed, best heat used
Skip penalty	15s per skip (cumulative n)	1st=15s, 2nd=30s, 3rd=45s ... devastating
Min track width	20 inches	Robot 16" → only 2" clearance each side
Heat duration	10 minutes	Multiple runs allowed per heat
Communication	None during run	RTK GNSS correction + E-Stop exempt

2.1 Architecture Style

The system follows a **layered, event-driven, publish-subscribe architecture** native to ROS2:

- **Layered:** 5 horizontal layers (Drivers, Localization, Perception, Navigation, Mission) with strict dependency direction -- upper layers consume lower layer topics only
- **Publish-Subscribe:** All inter-component communication uses ROS2 DDS topics — fully decoupled, zero direct function calls between layers
- **Event-Driven:** The Mission State Machine reacts to events (zone entry, obstacle classification, start signal) rather than polling
- **Dual-Compute:** Safety-critical path (E-Stop, motor control) runs on a separate processor (Raspberry Pi 4B) from the intelligence stack (Jetson Orin Nano)

2.2 Key Design Principles

● Fail-Safe by Default

If any component fails, the robot stops. E-Stop path never passes through ROS2. Motor commands timeout to zero if not refreshed within 100ms.

● Sensor Independence

No single sensor failure should make the robot crash. RTK loss: FAST-LIO2 continues. LiDAR degradation: reduce speed + use costmap memory. Camera failure: skip AprilTag detection, use LiDAR-only.

● Prior Map Advantage

Pre-recorded obstacle-free map gives instant initialization, pre-computed waypoints, and map differencing for obstacle detection. The robot never starts "blind."

● Minimal Custom Code

Use proven open-source packages (FAST-LIO2, Nav2, robot_localization). Custom code limited to: PCL classifier node, mission state machine, behavior switcher, CAN bridge. **Do not rewrite what exists.**

2.3 Constraints & Tradeoffs

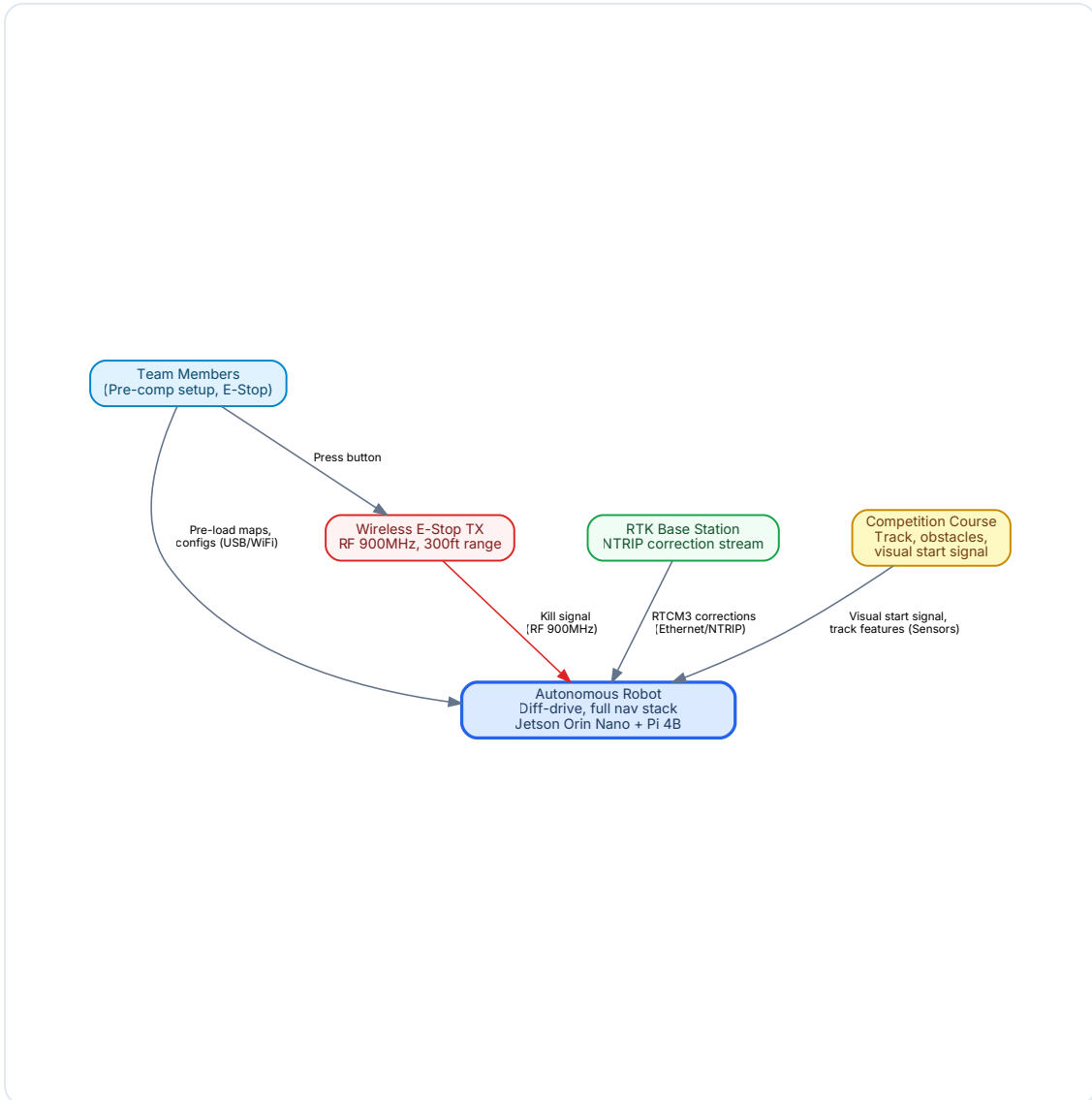
CONSTRAINT	IMPACT	TRADEOFF CHOSEN
16" width, 20" track	2" clearance — no room for lateral error	RTK + FAST-LIO2 fusion for <3cm lateral accuracy at narrow passages
12V / 15Ah battery	~180Wh. Orin Nano (15W) + QT64 (8W) + Pi (5W) + motors = ~50W steady, ~3.5h max	Sufficient for competition but no margin for extended testing. Bring charger.
Differential drive	Can turn in place (good) but skids on turns causing odometry slip	Do NOT use wheel odometry in EKF. FAST-LIO2 + RTK handles this without wheel data.
Outdoor in October	Rain, wind, sun glare possible	LiDAR primary (weather-resistant). Camera secondary only. No reliance on visual features.
Ethernet ports limited on Orin	QT64 (Ethernet) + BX992 (Ethernet) = need 2 Ethernet ports	USB-Ethernet adapter for second port, or Ethernet switch. Verify bandwidth.

CRITICAL TRADEOFF — NO WHEEL ODOMETRY

Differential-drive robots have severe wheel slip on turns (skid-steer). CAN-reported wheel velocities are physically inaccurate during any turning motion. Adding bad data to an EKF makes it worse. FAST-LIO2 (0.1–0.3% drift) + RTK (1cm) provides better odometry than any wheel encoder on a skid-steer platform.

System Context Diagram

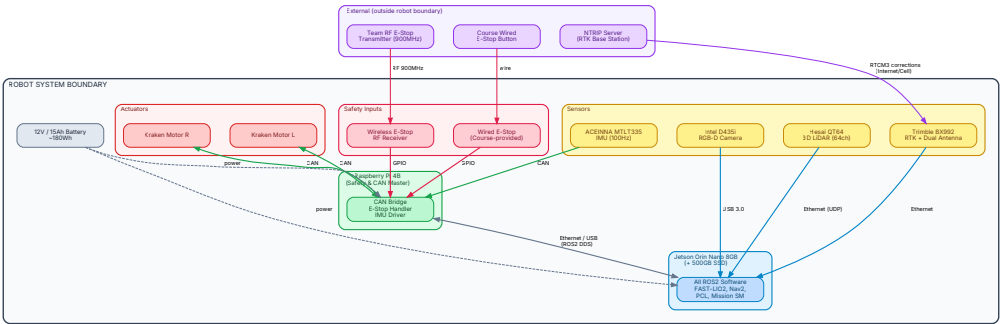
The following C4-style context diagram shows the robot system and its external interactions during competition.



External interactions during a competition run:

- **RTK Base Station** — Provides RTCM3 corrections via NTRIP/Ethernet to the Trimble BX992 receiver. This is the only allowed communication (Rule 2.4 exemption).
- **Competition Course** — The physical environment sensed by LiDAR, camera, and GNSS. Includes a visual start signal detected by D435i camera.
- **Wireless E-Stop** — Team member holds RF transmitter. E-Stop receiver on robot connects to Raspberry Pi GPIO. Signal loss = auto-stop.
- **Team Members** — No interaction during run. Pre-competition: load maps, calibration files, zone configs via USB or WiFi SSH.

3.1 System Boundary — Physical Interfaces



4.1 Hardware Components

COMPONENT	MODEL	INTERFACE	ROLE	NOTES
3D LiDAR	Hesai QT64	Ethernet (UDP)	FAST-LIO2 odometry, PCL obstacle detection, costmap	64ch, $\pm 45^\circ$ VFOV, 360° HFOV, $\sim 20\text{Hz}$. Front-mounted.
RGB-D Camera	Intel D435i	USB 3.0	Visual start detection, AprilTag calibration, depth assist	1080p/30fps. Built-in IMU NOT used as primary.
IMU	ACEINNA MTLT335	CAN	Primary IMU for FAST-LIO2. Internal Kalman correction.	MEMS, 100Hz output, pre-filtered. Industrial grade.
RTK GNSS	Trimble BX992	Ethernet	Global position (1cm) + GPS heading (dual-antenna)	Dual-antenna eliminates heading ambiguity at rest/low speed.
Compute — Primary	Jetson Orin Nano 8GB	—	All ROS2 software: perception, localization, navigation, mission	40 TOPS, 6-core ARM A78AE. 500GB SSD for rosbags.
Compute — Safety	Raspberry Pi 4B	Ethernet/USB to Jetson	CAN bus master (motors + IMU), E-Stop handler	MCP2515 SPI-CAN hat. Runs ROS2 or micro-ROS bridge.
Motors	Kraken $\times 2$	CAN (via Pi)	Left + Right differential drive	Wheel encoder data available but NOT used in EKF.
Battery	12V / 15Ah	Power bus	Single power source for all systems	$\sim 180\text{Wh}$. Estimated runtime: $\sim 3.5\text{h}$ at full load.
E-Stop (Wired)	Course-provided	GPIO $\rightarrow$ Pi	Hard safety stop (Rule 1.2.2)	Must stop robot within 1 second
E-Stop (Wireless)	RF 900MHz	GPIO $\rightarrow$ Pi	Team safety stop, 300ft range	Fail-safe: stop if signal lost $> 1\text{s}$

TRIMBLE BX992 — DUAL ANTENNA ADVANTAGE

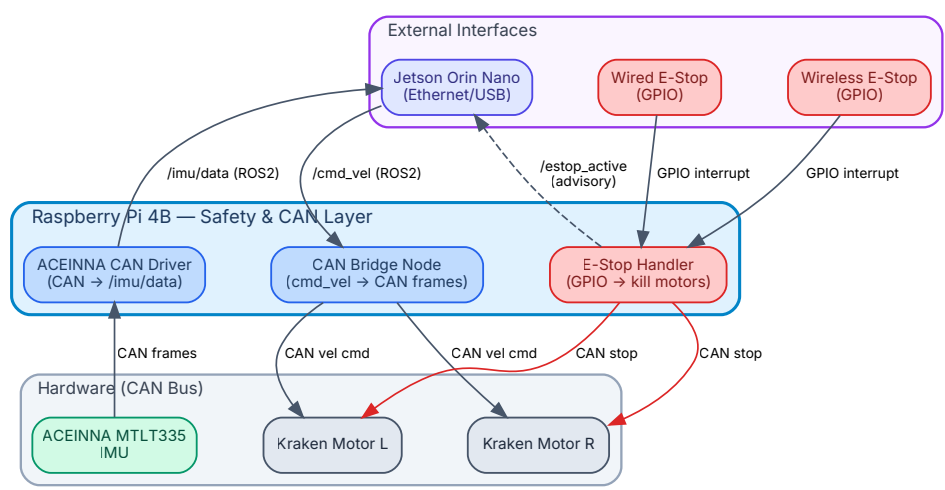
Unlike single-antenna RTK that cannot determine heading at standstill or low speed, the BX992 with two antennas computes heading from the baseline vector between them.

This provides accurate heading **from the moment the robot starts** — no need to drive forward first to compute heading from position deltas. Critical for competition where the robot must begin navigating immediately after start signal.

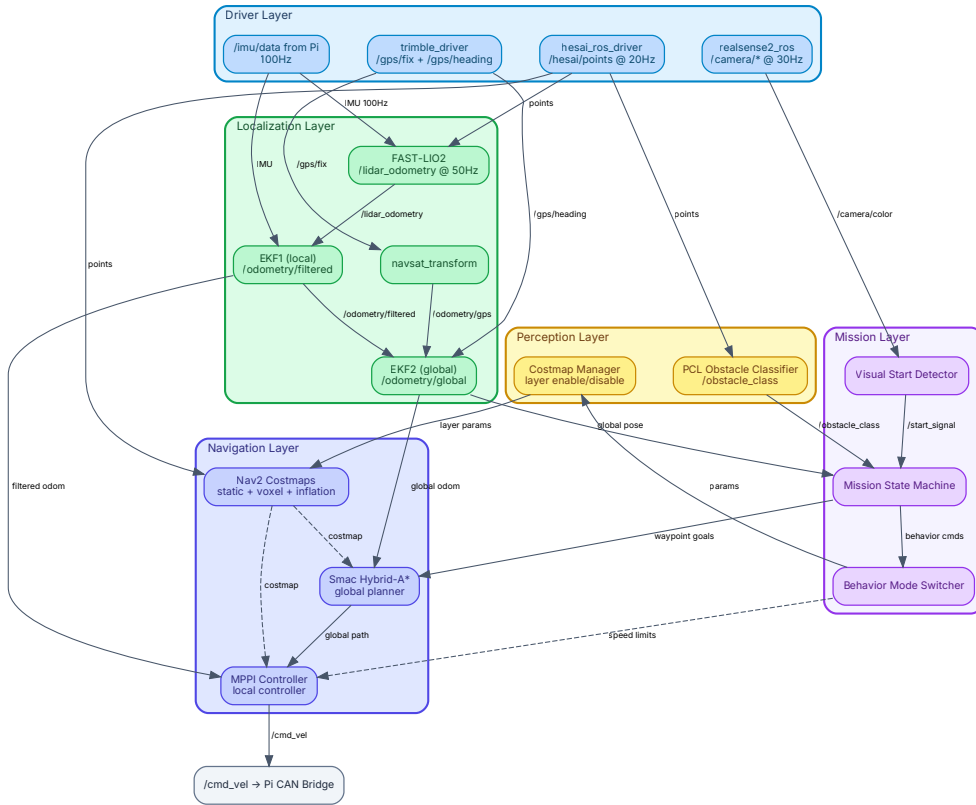
4.2 Software Component Diagram (UML)

The component architecture is split into two diagrams for readability: Part A shows the Raspberry Pi safety/CAN layer and its hardware connections; Part B shows the Jetson Orin Nano ROS2 stack and inter-layer data flows.

PART A — RASPBERRY PI 4B: SAFETY AND CAN LAYER



PART B — JETSON ORIN NANO: ROS2 HUMBLE STACK



Part A and Part B communicate over Ethernet/USB via ROS2 DDS. The Pi publishes `/imu/data` and subscribes to `/cmd_vel`. The E-Stop path on the Pi operates independently of ROS2 -- if Jetson or ROS2 fails, the Pi still stops the motors.

4.3 Component Descriptions

4.3.1 CAN BRIDGE NODE (RASPBERRY PI)

Purpose

Translates ROS2 `/cmd_vel` (Twist message) into CAN frames for the two Kraken motors. Also forwards ACEINNA IMU CAN frames as ROS2 `/imu/data` topic to the Jetson.

- **Subscribes:** `/cmd_vel` (geometry_msgs/Twist)
- **Publishes:** `/imu/data` (sensor_msgs/Imu) from ACEINNA via CAN
- **Hardware:** MCP2515 SPI-to-CAN hat on Raspberry Pi 4B
- **Diff-drive kinematics:** Converts linear.x + angular.z into left/right motor velocities
- **Watchdog:** If no `/cmd_vel` received for 100ms, sets motor velocity to 0

4.3.2 E-STOP HANDLER (RASPBERRY PI)

Purpose

Monitors both E-Stop inputs (wired GPIO + wireless RF GPIO). On activation OR wireless signal loss >1s, immediately sends zero-velocity CAN command to both motors — bypassing ROS2 entirely.

- **Input:** 2× GPIO pins (wired E-Stop, wireless RF receiver)
- **Output:** CAN motor stop command (direct, no ROS2 involvement)
- **Advisory:** Also publishes `/estop_active` (std_msgs/Bool) so Nav2 knows to pause
- **Latency requirement:** <1 second from trigger to motor stop (Rule 1.2.7.2)
- **Implementation:** Hardware interrupt on GPIO, highest priority process

4.3.3 FAST-LIO2 (JETSON)

Purpose

Primary odometry source. Tightly couples Hesai QT64 pointcloud with ACEINNA MTLT335 IMU to produce 50Hz ego-motion estimates via iterated Extended Kalman Filter.

- **Inputs:** `/hesai/points` (PointCloud2, ~20Hz) + `/imu/data` (Imu, 100Hz)
- **Output:** `/lidar_odometry` (nav_msgs/Odometry, ~50Hz)
- **Drift:** 0.1–0.3% of distance traveled (well-calibrated)
- **Key advantage:** Works in darkness (tunnel), unaffected by surface type, no wheel dependency
- **ACEINNA benefit:** MTLT335's internal Kalman pre-filters gyro bias → FAST-LIO2 gets cleaner IMU data → less drift

4.3.4 LOCALIZATION EKF STACK (JETSON)

● Purpose

Two-tier EKF using robot_localization package. EKF1 provides high-rate local odometry for the controller. EKF2 provides global drift-corrected pose for the planner.

INSTANCE	INPUTS	OUTPUT	FRAME	RATE
EKF1 (Local)	FAST-LIO2 odom + IMU angular vel	/odometry/ filtered	odom → base_link	50Hz+
EKF2 (Global)	EKF1 output + RTK position + RTK heading	/odometry/ global	map → odom	20Hz

Trimble BX992 heading advantage: EKF2 receives heading directly from dual-antenna GPS — no need for motion to estimate heading. Robot knows its exact orientation from the moment RTK fixes.

4.3.5 PCL OBSTACLE CLASSIFIER (JETSON)

● Purpose

Custom ROS2 node consuming QT64 pointcloud. Performs ground removal, clustering, map differencing, and per-cluster classification.

Pipeline (per scan, ~20Hz):

- 1. Ground removal: RANSAC plane fit → remove points within 5cm of plane
- 2. Clustering: pcl::EuclideanClusterExtraction (tolerance: 0.15m)
- 3. Map diff: Subtract prior_map.pcd → isolate NEW obstacles only
- 4. Classification: Per-cluster feature extraction → behavior assignment
 - └ CHIMES: thin, vertical, sparse → PUSH_THROUGH / BLIND_DRIVE
 - └ MOVABLE: dense, isolated, box-like → NAVIGATE_AROUND
 - └ TUNNEL: bounded walls both sides, open center → PUSH_THROUGH
 - └ WALL: dense, full-height, no gap → STOP + re-plan

Output: /obstacle_class (custom msg: zone_id, type, confidence, centroid)

4.3.6 NAV2 STACK (JETSON)

Purpose

- **Smac Hybrid-A* planner:** Non-holonomic path planning respecting robot kinematics. Even though diff-drive can turn in place, smooth curves are preferred for speed.
- **MPPI controller:** Samples 1000+ trajectory rollouts, selects lowest-cost. Smooth velocity commands, handles dynamic obstacles gracefully.
- **Costmap layers:** Static map (prior map OccupancyGrid) + Voxel (live QT64 points) + Inflation (robot footprint buffer)
- **Behavior switcher integration:** Mission layer can disable voxel layer for PUSH_THROUGH zones, preventing chimes from blocking the robot.

4.3.7 MISSION STATE MACHINE (JETSON)

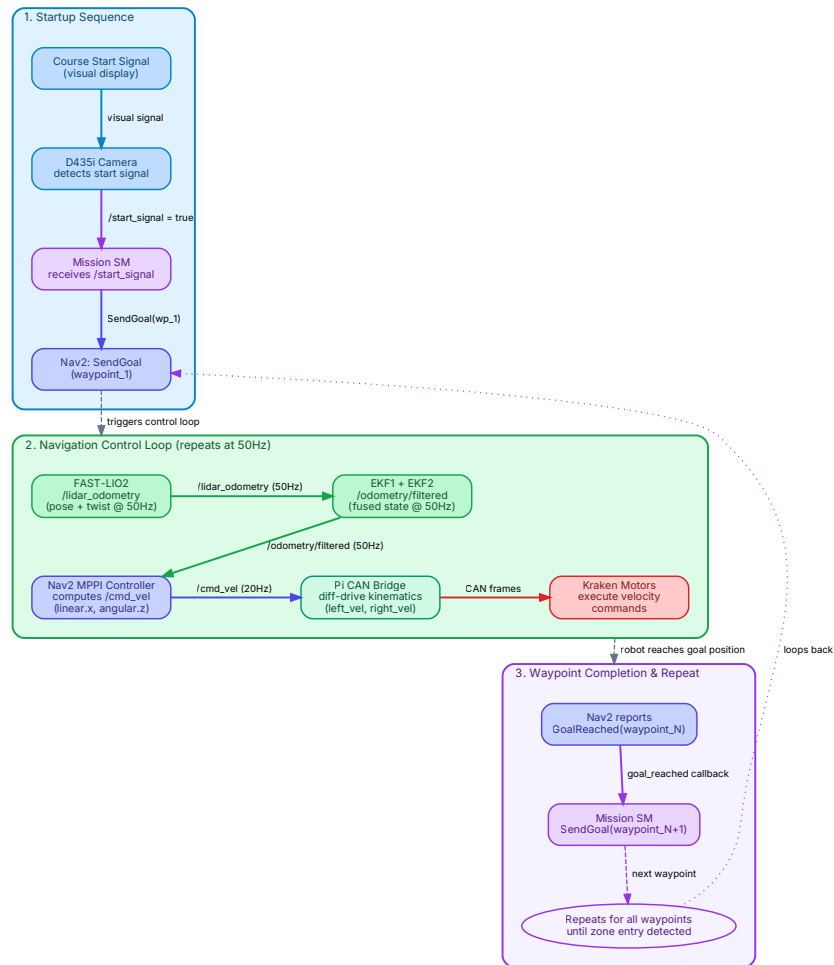
Purpose

Top-level controller that sequences the full competition run: wait for start → navigate waypoints → detect zones → switch behaviors → count laps → stop.

States:

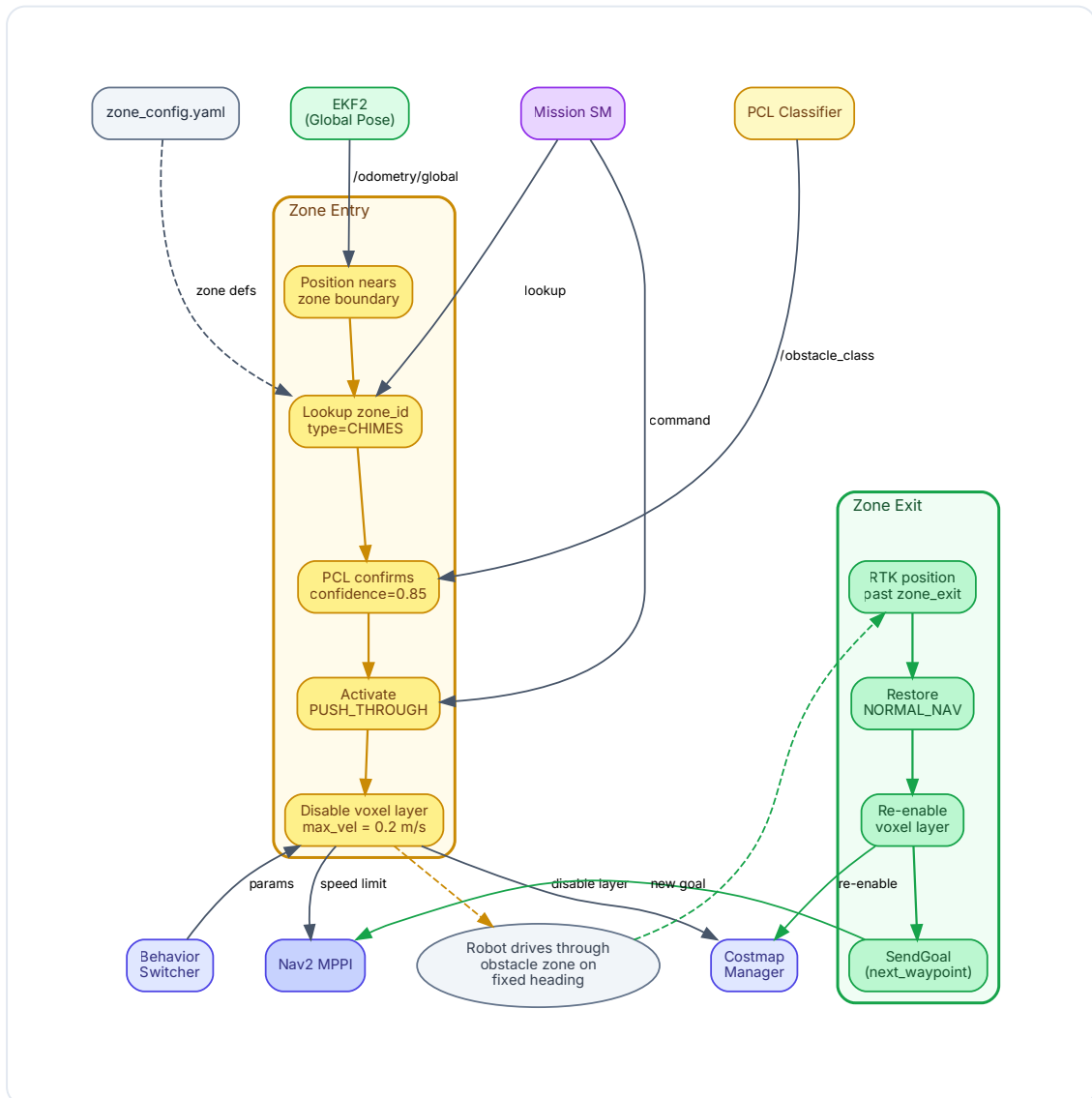
WAIT_FOR_START	→ monitor /start_signal from D435i start detector
NAVIGATE	→ send waypoint goals to Nav2, monitor zone proximity
ZONE_ENTRY	→ lookup zone_config.yaml, read /obstacle_class
BEHAVIOR_ACTIVE	→ activate mode (PUSH_THROUGH / NAVIGATE_AROUND / BLIND)
LAP_COMPLETE	→ increment counter, restart waypoints or STOP
STOP	→ /cmd_vel = 0, hold position
E_STOP	→ entered on /estop_active, resume on release

5.1 Normal Navigation Flow — Start to Waypoint



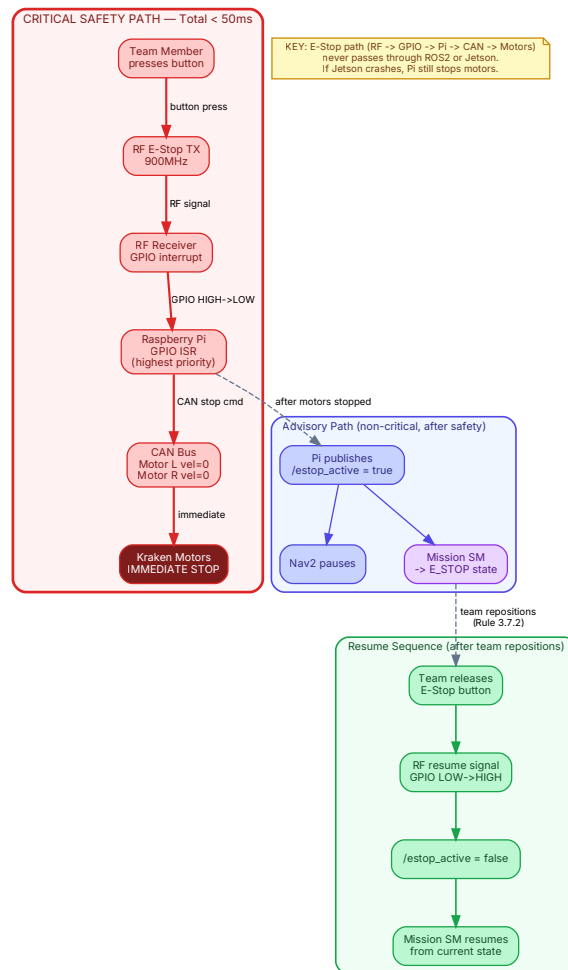
This sequence shows the steady-state navigation loop. FAST-LIO2 feeds the EKF at 50Hz, which feeds Nav2's controller, which publishes `/cmd_vel` consumed by the Pi's CAN bridge. The mission state machine sequences waypoints.

5.2 Obstacle Zone Entry — Behavior Switch



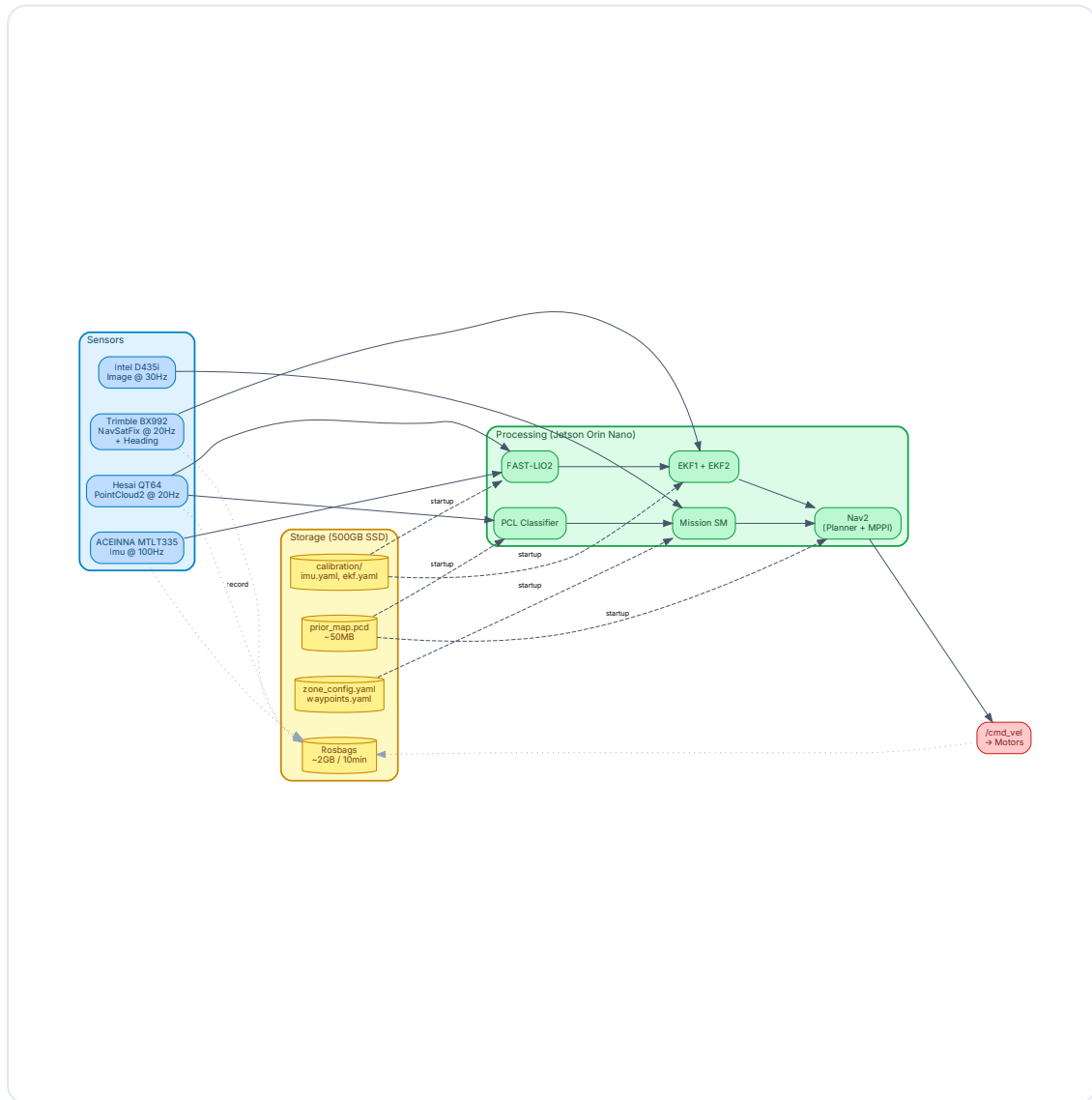
When the robot's global position approaches a zone boundary, the mission SM looks up the pre-configured behavior, confirms with live PCL classification, then activates the appropriate behavior mode. On zone exit (detected by RTK position), normal navigation resumes.

5.3 E-Stop Activation — Safety Path



Critical safety property: The E-Stop path (RF → GPIO → Pi → CAN → Motors) never passes through ROS2 or the Jetson. If the Jetson crashes, the Pi still handles E-Stop independently. The ROS2 notification is advisory only — it helps Nav2 behave gracefully but is not required for safety.

6.1 Data Flow Diagram



Solid arrows = real-time data flow during competition. Dashed arrows = offline/startup data loading and rosbag recording.

6.2 Storage & Configuration

FILE SYSTEM LAYOUT (500GB SSD ON JETSON ORIN NANO)

```
/home/robot/
├─ diy_ws/                                # ROS2 workspace
│   ├── src/
│   │   ├── fast_lio/                    # FAST-LIO2 package
│   │   ├── diy_perception/              # PCL classifier node
│   │   ├── diy_mission/                 # Mission state machine
│   │   ├── diy_bringup/                 # Launch files, configs
│   │   └─ can_bridge/                   # (built on Pi, shared source)
│   ├── build/
│   └─ install/
├─ maps/
│   ├── prior_map.pcd                    # 3D pointcloud (LIO-SAM output)
│   ├── static_map.pgm + .yaml           # 2D OccupancyGrid for Nav2
│   └─ waypoints.yaml                    # Centerline waypoints
├─ config/
│   ├── zone_config.yaml                 # Obstacle zones + behaviors
│   ├── nav2_params.yaml                 # Full Nav2 config
│   ├── ekf_local.yaml                   # EKF1 parameters
│   ├── ekf_global.yaml                  # EKF2 parameters
│   ├── fast_lio_config.yaml             # FAST-LIO2 IMU params, extrinsics
│   └─ calibration/
│       ├── imu_intrinsic.yaml            # ACEINNA noise params
│       ├── lidar_imu_extrinsic.yaml      # QT64 ↔ MTLT335 transform
│       ├── camera_lidar_extrinsic.yaml
│       └─ gps_lever_arm.yaml             # BX992 antenna offset
├─ rosbags/
│   ├── calibration_runs/
│   ├── test_runs/
│   └─ competition/
└─ logs/
```

CONFIGURATION FILES — PURPOSE & FORMAT

FILE	FORMAT	GENERATED BY	CONSUMED BY
prior_map.pcd	PCD binary	LIO-SAM offline processing	PCL classifier (map diff), FAST-LIO2 (init)
static_map.yaml	YAML + PGM image	octomap_server → 2D projection	Nav2 map_server (static costmap layer)
waypoints.yaml	YAML (list of poses)	Voronoi centerline extraction script	Mission state machine (goal sequence)
zone_config.yaml	YAML	Day-before obstacle scan + classifier	Mission SM (zone boundaries + behaviors)
nav2_params.yaml	YAML	Manual tuning	Nav2 lifecycle nodes
fast_lio_config.yaml	YAML	Calibration procedures	FAST-LIO2 node

6.3 ROS2 Topic Map

TOPIC	TYPE	PUBLISHER	SUBSCRIBER(S)	RATE
<code>/hesai/points</code>	PointCloud2	hesai_ros_driver	FAST-LIO2, PCL Classifier, Costmap Voxel	20 Hz
<code>/imu/data</code>	Imu	Pi CAN Bridge	FAST-LIO2, EKF1	100 Hz
<code>/gps/fix</code>	NavSatFix	Trimble driver	navsat_transform	20 Hz
<code>/gps/heading</code>	Float64 or Imu	Trimble driver	EKF2	20 Hz
<code>/lidar_odometry</code>	Odometry	FAST-LIO2	EKF1	50 Hz
<code>/odometry/filtered</code>	Odometry	EKF1	Nav2 MPPI, EKF2	50 Hz
<code>/odometry/global</code>	Odometry	EKF2	Nav2 Planner, Mission SM	20 Hz
<code>/cmd_vel</code>	Twist	Nav2 MPPI (or joy_node)	Pi CAN Bridge	20 Hz
<code>/obstacle_class</code>	Custom	PCL Classifier	Mission SM	~5 Hz
<code>/start_signal</code>	Bool	Start Detector	Mission SM	Event
<code>/estop_active</code>	Bool	Pi E-Stop Handler	Mission SM, Nav2	Event
<code>/camera/color/image_raw</code>	Image	realsense2_ros	Start Detector	30 Hz

TF TREE (CRITICAL — MUST BE CORRECT)

```

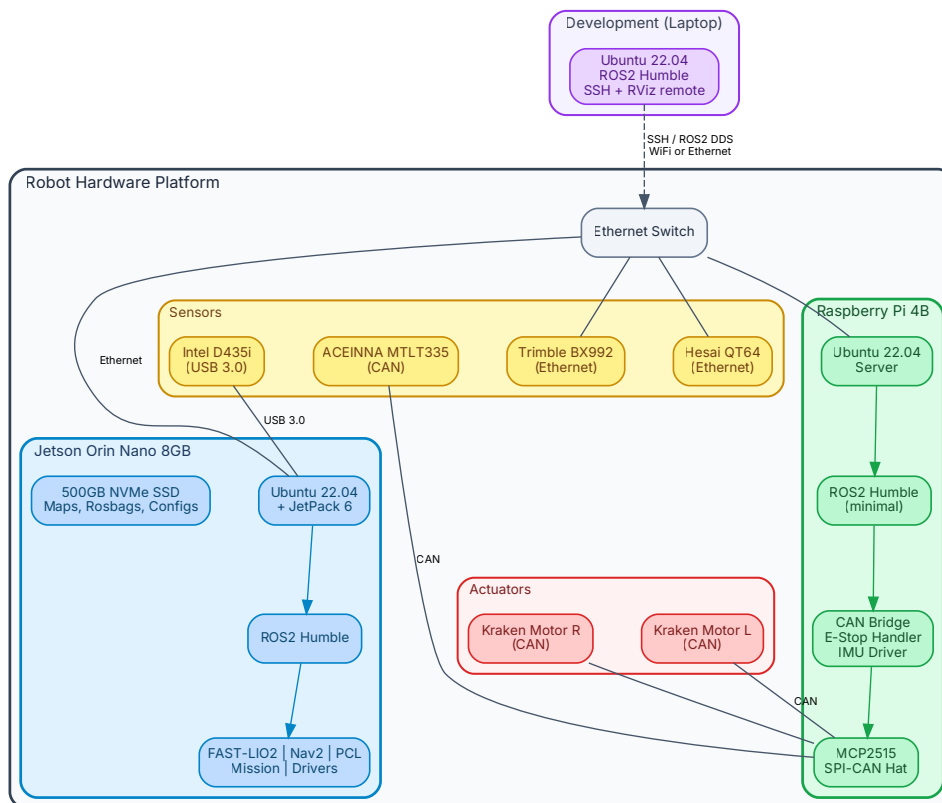
map
├── odom (published by EKF2 / navsat_transform)
│   └── base_link (published by EKF1)
│       ├── lidar_link (static: URDF, from lidar_imu_extrinsic.yaml)
│       ├── imu_link (static: URDF, from lidar_imu_extrinsic.yaml)
│       ├── camera_link (static: URDF, from camera_lidar_extrinsic.yaml)
│       ├── gps_link (static: URDF, BX992 antenna 1 offset)
│       └── gps2_link (static: URDF, BX992 antenna 2 offset)

```

CRITICAL: Get all static transforms into the URDF in week 1.

A missing or wrong transform = everything downstream fails silently.

7.1 Deployment Diagram (UML)



7.2 Environments

ENVIRONMENT	HARDWARE	PURPOSE	NETWORK
Development	Laptop (Ubuntu 22.04)	Code development, simulation, rosbag replay, RViz visualization	WiFi to robot or localhost
Robot — Lab	Full robot, indoors	Integration testing, calibration, unit tests on real hardware	Direct Ethernet to laptop
Robot — Field	Full robot, outdoors	Full autonomy testing, map recording, RTK verification	WiFi (monitoring only)
Robot — Competition	Full robot, competition venue	Competition heats. No laptop connected during run.	RTK NTRIP only

7.3 Software Deployment Process

1 Develop on laptop

Write/edit ROS2 packages in `diy_ws/src/`. Build with `colcon build`. Test with rosbag replay.

2 Deploy to Jetson via rsync

`rsync -avz diy_ws/src/ jetson:~/diy_ws/src/` then SSH and `colcon build` on Jetson.

3 Deploy to Pi via rsync

`rsync -avz can_bridge/ pi:~/diy_ws/src/can_bridge/` then build on Pi.

4 Launch full stack

`ros2 launch diy_bringup competition.launch.py` — brings up all nodes on both platforms via ROS2 launch system.

7.4 Power Budget

COMPONENT	STEADY DRAW	PEAK
Jetson Orin Nano (15W mode)	~12W	15W
Hesai QT64	~8W	12W
Trimble BX992	~5W	8W
Intel D435i	~3W	5W
Raspberry Pi 4B + CAN hat	~5W	7W
ACEINNA MTLT335	~2W	3W
Kraken Motors × 2 (avg drive)	~15W	60W
Total steady-state	~50W	~110W peak

Runtime estimate: 180Wh / 50W = 3.6 hours at steady state. Adequate for full competition day (heats are 10 min each, ~6 heats max across both courses).

8.1 Performance

METRIC	REQUIREMENT	HOW ACHIEVED
Localization accuracy	<3cm lateral at narrow passages	RTK (1cm) + FAST-LIO2 fusion. BX992 heading at standstill.
Odometry rate	≥50Hz to Nav2 controller	FAST-LIO2 outputs at 50Hz natively
Obstacle classification latency	<200ms from scan to decision	PCL pipeline single-threaded ~50ms per scan on Orin
E-Stop response	<1 second (Rule 1.2.7)	GPIO interrupt → CAN stop: ~50ms typical
Navigation replanning	<500ms for new global path	Smac Hybrid-A* on Orin Nano: ~100–300ms typical
Lap time target	<3 minutes per lap	MPPI controller at 0.5–1.0 m/s, zone slowdown to 0.2 m/s

8.2 Reliability

FAILURE MODE	DETECTION	RECOVERY
RTK fix lost (building shadow)	GPS status field → NO_FIX	FAST-LIO2 continues. EKF2 coasts on last fix. Drift ~5cm/10m.
LiDAR driver crash	No /hesai/points for 500ms	Mission SM → reduce speed to 0.05m/s, attempt driver restart. If fails → E-Stop.
Jetson crash/freeze	Pi watchdog: no /cmd_vel for 200ms	Pi auto-stops motors. Robot is safe. Team resets.
Pi crash/freeze	Motors receive no CAN for 100ms (motor controller watchdog)	Motor controllers have built-in timeout → coast to stop.
Single motor failure	CAN error frame or encoder feedback anomaly	Cannot continue — robot will spin. E-Stop and DNF.

8.3 Scalability

Not a traditional scalability concern (single robot, fixed hardware). However:

- **Compute headroom:** Orin Nano at 40 TOPS + 6 cores provides ~60% headroom over requirements. Can add additional perception (ML object detection) if needed without hardware changes.

- **Storage:** 500GB SSD allows recording every test session rosbag without cleanup. At ~2GB/10min, that's ~250 hours of recording.
- **Sensor upgrade path:** Adding sensors only requires new ROS2 driver + topic. Architecture is sensor-count agnostic.

8.4 Security

CONCERN	MITIGATION
Unauthorized access to robot	SSH key-only auth. WiFi disabled during competition runs. No open ports.
ROS2 DDS spoofing	DDS domain ID set to unique value. No network during run (Rule 2.4). Only RTK NTRIP allowed.
E-Stop RF jamming	Fail-safe design: signal loss = stop. Jamming cannot prevent stopping.
NTRIP stream corruption	Trimble BX992 validates RTCM3 checksums. Invalid corrections rejected internally.

8.5 Observability

TOOL	PURPOSE	WHEN USED
RViz2 (remote)	Visualize costmap, path, pointcloud, TF tree, robot pose	Development + pre-competition. NOT during run.
Rosbag recording	Record all topics for offline replay and debugging	Every drive session including competition
ros2 topic echo	Spot-check topic values	Debugging
PlotJuggler	Time-series visualization of EKF drift, motor commands, IMU data	Post-run analysis from rosbags
LED status indicator	Simple RGB LED showing robot state (WAIT=blue, NAV=green, ZONE=yellow, ESTOP=red)	Competition — visible to team at distance
journalctl	System logs for node crashes, resource issues	Post-run debugging

9.1 Technical Risks

RISK	LIKELIHOOD	SEVERITY	MITIGATION
ACEINNA CAN protocol unknown	High (now)	High	Obtain MTLT335 datasheet immediately. Verify CAN frame format. If proprietary, may need vendor SDK or custom parser.
Kraken CAN protocol undocumented	Medium	High	Identify motor controller type (CTRE Phoenix? REV?). Get CAN protocol docs. Sniff existing CAN traffic if no docs.
Ethernet port contention (QT64 + BX992)	Medium	Medium	Use USB-Ethernet dongle or 5-port unmanaged switch. Verify no packet loss under load.
Chimes block robot (classified as wall)	Medium	High	BLIND_DRIVE_MODE fallback — disable all obstacle detection, drive straight for zone_width distance.
Differential-drive slips on gravel turns	High	Low	No wheel odometry in EKF. FAST-LIO2 unaffected by wheel slip. Reduce turn speed on gravel surfaces.
MCP2515 CAN bus throughput limit	Low	Medium	MCP2515 max: 1Mbps. With 2 motors + IMU at 100Hz: ~30kbps. Well within limits.
Trimble BX992 Ethernet config complexity	Medium	Medium	BX992 outputs NMEA or proprietary binary. Verify ROS2 driver compatibility (nmea_navsat_driver or custom). May need TCP socket parser.
Weight exceeds 25lbs after adding sensors	Low-Medium	High	Weigh robot after each sensor addition. BX992 + antenna + Pi + CAN hat ≈ 1.5lbs total. Verify headroom.
Battery not 12V regulated for all components	Medium	High	Need voltage regulators: 5V for Pi, 5V for D435i USB. Jetson Orin Nano needs specific voltage (check datasheet — may need carrier board with regulator).
TF tree errors found late	Low (if caught early)	High	Build URDF in week 1. Run <code>tf2_echo</code> on all sensor pairs. Verify in RViz before any odometry work.

9.2 Schedule Risks

RISK	IMPACT	MITIGATION
CAN protocol reverse-engineering takes weeks	Blocks all motor control + IMU integration	Start CAN sniffing in week 1. If no docs in 2 weeks, escalate to vendor support.
FAST-LIO2 ROS2 port has compatibility issues	Blocks localization stack	Test FAST-LIO2 build on Orin Nano with QT64 driver in first 2 weeks. Have backup: LIO-SAM online mode.
Trimble driver not available for ROS2	Blocks RTK integration	Worst case: write a simple TCP socket → NavSatFix publisher (NMEA parsing is straightforward).
Team member unavailable for extended period	Delays dependent work	Document everything. This SAD + code comments + rosbags allow anyone to continue.

9.3 Competition Day Risks

RISK	MITIGATION
Rain/wind on competition day	LiDAR primary (rain-resistant at low intensity). Camera secondary only. Reduce speed. Cover electronics with weatherproof enclosure.
RTK base station has no cell service at venue	Bring own mobile hotspot for NTRIP. Or bring portable RTK base station if available.
Robot repositioned after E-Stop (Rule 3.7.2)	FAST-LIO2 re-localizes against prior map. RTK resumes within 30s. Publish manual initial pose if needed.
Obstacle moved between laps by previous team	Re-classify at every zone entry (never cache lap-1 results for lap-2). Map diff handles displaced objects.

HIGHEST PRIORITY RISK ITEMS (RESOLVE THIS WEEK)

1. **Kraken motor CAN protocol:** What controller? What CAN frame format for velocity commands?
2. **ACEINNA MTLT335 CAN protocol:** What DBC/frame IDs output orientation + angular velocity?
3. **Trimble BX992 output format:** NMEA over TCP? Binary? What port/protocol for heading?
4. **Power distribution:** How does 12V battery feed Jetson Orin, Pi, sensors? What regulators exist on chassis?



May 18 → June 1, 2026

Foundation — Drivers + CAN + TF

- Resolve CAN protocols for Kraken motors + ACEINNA MTLT335
- Pi 4B: MCP2515 hat, CAN bridge node, E-Stop handler
- Jetson: QT64 driver running, D435i driver, Trimble driver
- URDF complete with all sensor transforms
- TF tree verified in RViz (all frames publishing)
- FAST-LIO2 building and running on QT64 + ACEINNA IMU data
- Design review submission: this SAD + hardware photos + FAST-LIO2 demo



June 1 → June 30, 2026

Localization + Calibration

- EKF1 + EKF2 running with Trimble BX992 (position + heading)
- IMU intrinsic calibration (ACEINNA may not need — internal Kalman. Verify.)
- LiDAR-IMU extrinsic calibration
- Camera-LiDAR extrinsic calibration
- GPS lever arm measurement
- Verify lap drift <10cm in open-sky test area
- All hardware purchases submitted by June 30 deadline



July → August 14, 2026

Path Following Demo (Rule 0.2.4)

- Prior map recorded (joystick + LIO-SAM offline)
- Nav2 stack running: Smac + MPPI + costmap layers
- Robot completes clean 3-lap loop autonomously
- Visual start detection working
- E-Stop integration tested (both wired + wireless)
- Demo video: 3 laps at competition speed, zero human intervention



August → September 23, 2026

Obstacle Stack + Safety Review

- PCL classifier running on all obstacle types
- Behavior switcher validated per obstacle
- Map differencing pipeline end-to-end
- zone_config.yaml workflow tested
- Safety review: E-Stop timing measured, flag mount, documentation
- EKF tuning using AprilTag ground truth sessions



Sep 23 → Oct 1, 2026

Competition Prep + Dry Run

- Full 2-lap obstacle course simulation
- Time each obstacle, measure consistency
- MPPI parameter tuning for speed/precision
- Battery life verification for 3+ consecutive heats
- Pack competition kit: charger, laptop, spare configs, tools



October 1-2, 2026

Competition

- Day 0: obstacle scan lap, map diff, zone_config.yaml, upload, RViz verify
- Morning: RTK fix verify (~5min), practice heat
- Heats: trust the system. Do not change params between heats unless specific measurable issue identified.
- Record rosbag of every heat.